

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Graph-aware Large Language Models (LLMs) are emerging as alternatives to Graph Neural Networks (GNNs) for graph tasks such as node classification, but their adversarial robustness remains largely unmeasured. This paper makes five contributions toward closing that gap. First, we provide the first systematic robustness study of projector-based graph-aware LLMs by transferring representative GNN adversarial attacks, including poisoning and evasion attacks, to two representative models, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). Second, we organize graph-aware LLM methods through a graph-encoding taxonomy that separates textual-description approaches from learned-projector approaches, clarifying why projector-based models expose graph-specific robustness questions. Third, we identify a new LLAGA attack surface: malicious nodes can be injected as placeholders in the node sequence template, which can severely degrade performance. Fourth, our systematic analysis shows that graph encoding choices shape attack success: the node sequence template in LLAGA increases vulnerability, the GNN encoder used in GRAPH PROMPTER demonstrates greater robustness, and both approaches remain susceptible to imperceptible textual feature perturbations. Fifth, we propose GALGUARD, an end-to-end defense framework that combines an LLM-based feature correction module with adapted GNN defenses for structural attacks. Together, these results show that graph-aware LLM robustness depends on the graph encoding interface, the balance between textual features and graph structure, and defenses that address both channels simultaneously¹

1 Introduction

Graphs encode entities and relationships through nodes and edges, making them central to applications in social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and health-care (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018) have long been the standard modeling choice for graph tasks such as node classification and link prediction. At the same time, Large Language Models (LLMs), including GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), have motivated a new family of

graph-aware LLMs (Xie et al. 2023) that adapt language models to graph-structured inputs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Because recent graph-aware LLMs can outperform GNNs on several graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), their robustness is no longer a peripheral concern: if these models become practical replacements for GNNs, reviewers and practitioners need to know whether existing graph attacks transfer and where new vulnerabilities arise.

The first contribution of this paper is to make that robustness question explicit for graph-aware LLMs. Existing GNN adversarial attacks include poisoning attacks at training time and evasion attacks at test time (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023). These attacks usually assume black-box access and compromise *model integrity* by subtly modifying graph structure or node features. However, the graph-aware LLM setting changes the attack surface: the model still consumes graph structure, but it also relies on textual node features and a graph-to-language encoding interface. Prior GNN robustness work has emphasized structural attacks, whereas graph-aware LLMs make imperceptible feature perturbations a first-class risk because small textual changes can alter the information passed into the LLM.

These observations lead to our central question: *Do adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm, and what new vulnerabilities appear when graphs are encoded for LLMs?* Answering this question requires more than applying attacks to a new model class. The success of a graph-aware LLM depends heavily on how the graph is integrated into the language model. Node features can be handled naturally as text, but graph structure must be encoded through prompts, templates, learned projectors, GNN encoders, or related mechanisms (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore first contribute a graph-encoding taxonomy that separates approaches representing graphs as *textual descriptions* from approaches using *learned projectors* to bridge graph and text domains. Since conventional text attacks are more directly applicable to textual-description methods, our experiments focus on learned-projector methods with

¹Our code will be released upon acceptance.

frozen pre-trained LLMs: LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024).

Our study surfaces three robustness contributions that are useful for assessing this model family. First, transferring representative GNN attacks shows that graph encoding choices affect attack success: node-sequence projectors such as LLAGA are more exposed to structural perturbations, whereas GNN-based encoders can use unchanged textual embeddings to partially buffer structural attacks. Second, we identify a model-specific attack surface where malicious placeholders corrupt the node sequence representation supplied to the projector. Third, we show that imperceptible textual feature perturbations can be highly damaging, often exceeding the effect of structural perturbations and revealing a vulnerability that is understated in conventional GNN evaluations. To move from diagnosis to mitigation, we also propose graph-aware LLM defense, GALGUARD, which combines an LLM-based feature corrector with adapted GNN-based structural defenses. Our contributions are:

- **Transfer study of GNN attacks.** We adapt representative adversarial attacks for GNNs, specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to evaluate poisoning and evasion vulnerabilities in graph-aware LLMs.
- **Graph-encoding taxonomy.** We organize graph-aware LLMs by how they encode graph structure for LLM consumption, clarifying why learned-projector methods expose graph-specific attack surfaces.
- **New template-injection attack surface.** We discover a new attack surface in LLAGA, showing that injecting malicious nodes as placeholders into the node sequence template can degrade its performance.
- **Feature-perturbation robustness assessment.** We evaluate imperceptible attacks on textual node features and show that graph-aware LLMs can be more vulnerable to feature perturbations than traditional GNN-centered analyses would suggest.
- **Integrated defense framework.** We propose GALGUARD, an end-to-end defense strategy that combines LLM-based feature correction with adapted GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Our first framing contribution is a taxonomy that connects graph-aware LLM design choices to adversarial attack surfaces. We group graph encodings into two families: *textual descriptions of graphs*, where graph information is written as natural language, and *learned graph projectors*, where a trained projector maps graph data into embeddings or structured token sequences that a frozen LLM can process. Figure 5 illustrates the distinction. Appendix A.1 gives the full taxonomy, and Appendix A.2 reviews related work on GNN attacks.

Graph Representation as a Textual Description. Textual-description methods convert graph data into natural language

prompts or descriptions, allowing the LLM to operate on graph information through its native text interface (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) connect molecular graphs with textual descriptions. Other work studies graph-structured prompting directly: Huang et al. (2023a) show that LLMs often treat graph prompts as contextual paragraphs, which makes prompt design central in systems such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). A related thread uses LLMs to enhance node features, graph structures, predictors, or feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) combine LLMs and GNNs for textual graphs, and Duan et al. (2023) improve textual graph learning with LLM-derived representations. These methods are important background, but their robustness questions largely inherit the familiar text-attack interface and are constrained by prompt scalability and structural fidelity.

Graph Representation as a Learned Projector. Learned-projector methods address the limits of textual graph descriptions by transforming graph data into embeddings or graph vectors before LLM processing. This family includes LLAGA, which uses node-level templates and a linear projector (Chen et al. 2024), as well as methods that incorporate GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Other examples use graph neural prompts for knowledge graphs (Tian et al. 2024), prompt-based node feature extractors with GNN adapters (Huang et al. 2023b), or fixed-length graph prefixes produced by graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). This category is central to our contribution because the projector interface can amplify or dampen structural perturbations in ways that are not visible from the LLM alone.

This Work: We focus on projector methods with *frozen pre-trained LLMs* because they are computationally practical and avoid the cost of retraining the full language model (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Our two representative models are LLAGA (Chen et al. 2024), which uses node-level templates with a linear projector, and GRAPHPROMPTER (Liu et al. 2024), which uses a GNN with a linear projector. We exclude textual-description methods from the main experiments because their attacks are closer to standard text adversarial attacks and because they do not expose the graph-specific projector vulnerabilities that this paper aims to characterize.

3 Threat Model

Our threat model contributes a reviewer-facing bridge between established graph adversarial settings and graph-aware LLM deployment. Following prior work on adversarial graph attacks (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023), we study attackers with limited but realistic influence over graph data. In social networks, such an attacker may control

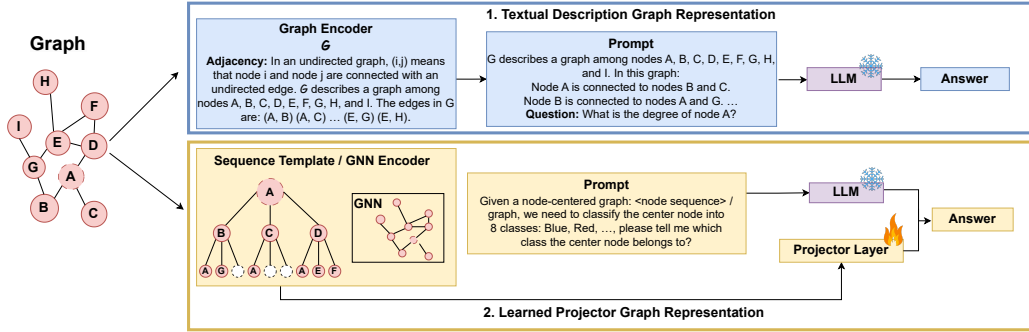


Figure 1: **Graph encoding-based adaptations of LLMs.** The taxonomy used in this paper separates graph-aware LLMs by the encoding interface that creates their robustness profile: textual-description methods express graphs in natural language (top), while learned-projector methods encode graphs through node-template sequences or GNN encoders before mapping them into LLM token space (bottom). In both projector cases, the LLM remains frozen and only the graph-to-token projector is trained.

a small number of bot accounts to evade detection or influence the classification of legitimate users. In Wikipedia-like settings, hoax articles with sparse links to legitimate articles can be connected strategically to affect category prediction for real content. These scenarios matter because graph-aware LLMs may be deployed in recommendation, social media analysis, healthcare, and other settings where they could replace GNNs. We therefore define the threat model along four explicit dimensions: model access, adversarial capabilities, attack strategies, and the attacker’s goal.

Model Access. The attacker operates in a black-box setting and has no access to the target model’s architecture, parameters, gradients, or internal representations. The attacker can influence the model only through training or test data.

Adversarial Capabilities. Consistent with existing poisoning and evasion attacks, the attacker has access to the training data for poisoning and to the inference data for evasion. Under a predefined budget and unnoticeability constraints, the attacker can modify or inject data while attempting to avoid detection.

Attack Strategies. The attacker uses two channels that correspond to the main contributions of our evaluation: feature manipulation, which alters textual node features, and structure manipulation, which adds or removes graph edges.

Attacker’s Goal. The attacker aims to compromise model integrity by degrading overall node-classification performance, measured through an increased misclassification rate.

4 Experimental Setup

Dataset. Our experimental design supports the paper’s robustness contributions by evaluating attacks on three standard text-attributed graph benchmarks: Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016). These datasets span small to medium graph sizes and different sparsity levels, and they are commonly used to evaluate graph-aware LLMs related to our study (Chen et al. 2023a, 2024; Liu et al. 2024). To check whether the template-injection finding scales beyond these benchmarks, we also report an additional ArXiv experiment (Hu et al. 2020) in Appendix A.5. We follow the standard train/validation/test splits of

6:2:2 for Cora, Citeseer, and PubMed, and 5:2:3 for ArXiv, as in (Chen et al. 2024). Table 1 summarizes the benchmark statistics used for the main experiments.

Table 1: Dataset statistics for the main robustness evaluation.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We use *node classification* as the evaluation task because it is the primary benchmark for both graph machine learning models and graph adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023). This choice keeps the comparison aligned with GNN robustness work while exposing how the graph-aware LLM encoding interface changes attack behavior.

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted labels, so that clean performance and attack-induced degradation are directly comparable across models, datasets, and attack types. Each experiment is repeated three times, and we report the mean and standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory. **Choice of Graph-aware LLM and Adversarial Attack Methods.** This setup section makes the experimental contribution explicit: we choose representative graph-aware LLM encoders and representative GNN attacks so that the results isolate whether robustness changes with the graph-to-LLM interface.

Graph-Aware LLM Methods. We evaluate two learned-projector graph-aware LLMs, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), because they instantiate different graph encoding choices while keeping the pre-trained LLM frozen. LLAGA uses node-level templates followed by a linear projector, whereas GRAPHPROMPTER uses a GNN-based encoder followed by a linear projector. In

Table 2: Transferred GNN structural attacks on graph-aware LLMs. The table reports NETTACK and METAATTACK performance on Cora, Citeseer, and PubMed for LLAGA and GRAPHPROMPTER, with percentage drops from clean accuracy in parentheses.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

both systems, the projector is a 2-layer multi-layer perceptron (MLP). The foundation LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We use poisoning and evasion variants of two representative GNN adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to test transferability into graph-aware LLMs. NETTACK is a targeted attack that searches for minimal perturbations causing specific-node misclassification, while METAATTACK is an untargeted attack that optimizes graph structure as a hyperparameter. To use NETTACK in an untargeted evaluation, we follow (Li et al. 2023; Zügner and Günnemann 2019): each test node is targeted sequentially and the results are aggregated. Across experiments, we perturb only 10% of the edges to preserve the unnoticeability constraint used in prior work (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Appendix A.2 provides additional attack details.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

Our first result section tests the transfer contribution directly: we adapt NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to poisoning and evasion settings for graph-aware LLMs. In poisoning attacks on LLAGA, the perturbed graph is used to generate node-level sequences before retraining the projector. In poisoning attacks on GRAPHPROMPTER, the perturbed graph is re-encoded with a GAT before retraining the projector. In evasion attacks, both models are trained on clean data and receive perturbations only during inference.

Results. Table 2 supports three main findings. First, poisoning attacks transfer only moderately to these graph-aware LLMs, with a maximum degradation of 11% across the evaluated structural attacks; this indicates that the learned projector and frozen LLM context can absorb some training-time structural noise. Second, evasion attacks are substantially more damaging, reaching a 51% performance drop and showing that graph-aware LLMs remain vulnerable when perturbed structure is introduced at inference time. Third, METAATTACK generally outperforms NETTACK across poisoning and evasion settings, consistent with its ability to optimize small

but effective edge perturbations by treating graph edges as hyperparameters. Dataset density also matters: LLAGA drops by 51% on denser Cora under evasion, but by 19% on sparser PubMed.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Table 2 clarifies why graph encoding is a central contribution of the paper: LLAGA is more susceptible than GRAPHPROMPTER due to its *sensitivity to structural perturbations*. The node-template method integrates graph structure by generating a fixed-length node-level sequence for each node and by computing a graph Laplacian embedding. Each sequence position corresponds to a relative structural position in the original graph, so perturbed structure directly changes the information supplied to the LLM. In contrast, GRAPHPROMPTER’s textual node embeddings remain unchanged when structure is perturbed because only the GNN component receives the structural perturbation. This separation lets unperturbed textual features compensate for corrupted graph structure, improving robustness.

6 Node Sequence Template Injection Attacks

This section presents a new attack contribution that arises from the graph encoding design. The neighborhood detail template constructs node sequences through a fixed-shape computational tree, and that fixed shape creates placeholder positions that an attacker can exploit. Figure 2 illustrates the attack, and Appendix A.4 provides the algorithm. For a target node u , LLAGA samples 1-hop neighbors as children in the tree and repeats the process for subsequent levels. When a node has fewer than the required number of children, placeholders are inserted to maintain a fixed size such as $k = 10$, following LLAGA’s original design (see “Clean” in Figure 2 and lines 1-14 in the algorithm). Those placeholders become a graph-specific attack surface.

We instantiate this attack surface with three placeholder-injection strategies. *Non-Adjacent Injection (NI)* inserts nodes that are outside the 2-hop neighborhood of target node u (see “Attacked” in Figure 2 and lines 21-27). *Supernode Injection (SI)* inserts a high-degree node outside the target node’s 2-hop neighborhood into a placeholder. *Multiple Supernode Injection (MSI)* extends SI by inserting multiple distinct supernodes into placeholder positions (lines 35-43). The mech-

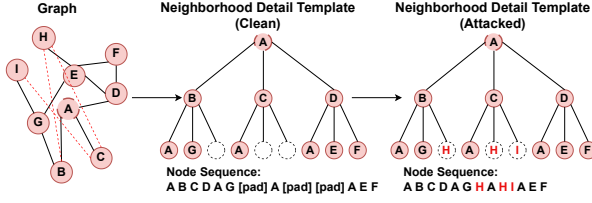


Figure 2: Node sequence template injection attack surface in LLAGA. The example shows Non-adjacent Injection (NI), where red dashed edges denote attacker-added perturbations. In Supernode Injection (SI) and Multiple Supernode Injection (MSI), one or more supernodes are inserted into placeholder positions. Here, $k = 3$.

Table 3: Node sequence template injection results for LLAGA. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean reports accuracy without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 $\pm$ 0.07	—
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	—	0.71 $\pm$ 0.04	—
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	—	0.90 $\pm$ 0.04	—
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03

anism is simple but important: uninformative injected nodes contaminate the node sequence, corrupt the target representation, and weaken the projection layer used for learning and inference.

Results. As shown in Table 3, LLAGA shows high resilience to poisoning attacks on the Cora and Citeseer datasets, but suffers a significant performance drop during evasion, with accuracies decreasing to 0.42, 0.36, and 0.30 for Cora and to 0.36, 0.30, and 0.24 for Citeseer on the *NI*, *SI*, and *MSI* attacks respectively.

PubMed shows a different robustness profile: evasion accuracies remain 0.85, 0.82, and 0.76 under *NI*, *SI*, and *MSI*. This contrast suggests that the contribution is topology-dependent. In smaller, more tightly connected graphs such as Cora and Citeseer, a localized placeholder injection can heavily distort the neighborhood representation. In the larger PubMed graph, the same type of injection is diluted and causes less severe degradation.

Relative to traditional structural attacks, placeholder injection can be more damaging for LLAGA. For example, *MSI* yields evasion accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, compared with METAATTACK accuracies of 0.44, 0.50, and 0.73 (Table 2). This comparison highlights why model-specific graph encoders deserve separate robustness analysis.

7 Poisoning and Evasion Attack via Feature Perturbation

Our next contribution shifts the robustness lens from graph structure to textual node features. Most graph adversarial attacks focus on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019); NETTACK (Zügner, Akbarnejad, and Günnemann 2018) also perturbs features, but its feature representation is not directly transferable to textual node attributes. Because structural attacks are only moderately successful against models such as GRAPHPROMPTER and because textual features strongly influence graph-aware LLM predictions, we evaluate unnoticeable feature perturbation attacks as a separate vulnerability channel.

We adopt imperceptible attack strategies from (Boucher et al. 2022) with one modification that makes the attack task-agnostic: perturbations are generated through an English-to-French translation objective. The attacker changes the input so that it appears visually identical or nearly identical to a human reader while inducing an incorrect translation, which indicates corrupted encoded semantics. We evaluate two perturbation types: *Homoglyphs*, which replace characters with visually similar alternatives, and *Reorderings*, which use direction-control characters to alter rendering order. Appendix A.6 gives examples of both attacks.

Following (Boucher et al. 2022), we optimize perturbations with differential evolution (Storn and Price 1997). The objective maximizes the Levenshtein distance between model outputs on perturbed and unperturbed text, and the perturbation budget is fixed at 10% of the average length of each textual feature.

Results. Table 4 shows that feature perturbations are a major vulnerability for graph-aware LLMs, with Reordering attacks generally stronger than Homoglyph attacks across models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both attacks: poisoning drops accuracy by 27% and 23% under Homoglyph, and by 35% and 25% under Reordering, respectively. GRAPHPROMPTER is more resilient in these settings. PubMed follows the same qualitative trend, although the impact is less pronounced than on the smaller datasets.

The feature contribution becomes clearer when compared with structural perturbations in Figure 3. Reordering attacks outperform structural METAATTACK perturbations by a large margin on both LLAGA and GRAPHPROMPTER. This indicates that graph-aware LLMs may rely more heavily on textual node features than on graph structure, making feature robustness central to any defense. The next section therefore combines structural and feature perturbations in a unified attack.

8 A Unified Imperceptible Attack on Graph-aware LLMs

The unified attack contribution tests whether the two vulnerability channels identified above compound. We combine feature and structural perturbations to form an adversarial strategy that remains inconspicuous while attacking both the relational and textual inputs of graph-aware LLMs. The hypothesis is that the combined attack degrades accuracy more

Table 4: Feature perturbation attack performance on graph-aware LLMs. Percentage drops from clean accuracy are shown in parentheses, highlighting the contribution of imperceptible textual attacks.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

Table 5: Unified imperceptible attack results. The table reports model accuracy when structural and feature perturbations are combined, with percentage drops from clean accuracy in parentheses.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

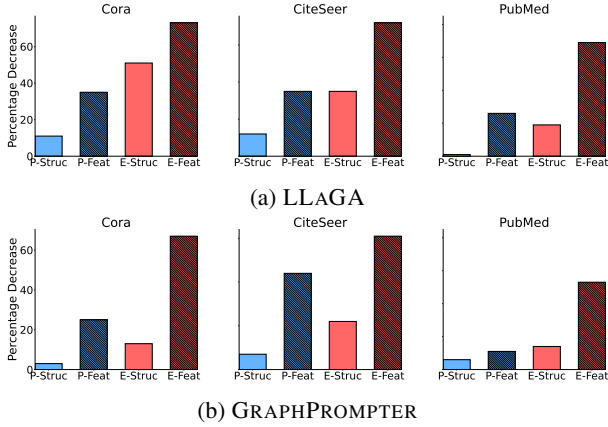


Figure 3: Structural versus feature attack salience for graph-aware LLMs. Attack success is measured as the percentage decrease from clean accuracy on the y-axis. P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning and evasion attacks based on structural and feature perturbations.

than either perturbation family alone.

For the structural component, we use METAATTACK to subtly modify graph edges. For the feature component, we combine Homoglyph and Reordering attacks from the previous section. This paired design targets the two information channels that our earlier results show to be important: graph relations and textual node features.

Results. Table 5 shows that jointly applying structural and feature perturbations is more damaging than either perturbation type alone across datasets. This result reinforces the paper’s central robustness claim: graph-aware LLMs differ from traditional GNNs because feature-level attacks can be as important as, or more important than, structural attacks. The unified attack succeeds because these models rely on consistency between graph structure and textual node features, so inconspicuous corruption of both channels sharply reduces accuracy.

9 Ablation Study

9.1 Varying Feature Perturbation

To characterize the sensitivity of the feature-attack contribution, we vary the percentage of perturbed textual features under the combined METAATTACK setting. Reordering follows the same qualitative pattern as Homoglyph, so we use it to summarize the trend from 1% to 10%. Across both representative models and the Cora, Citeseer, and PubMed datasets, reducing the perturbed-feature percentage weakens the attack. As the perturbation level decreases, poisoning and evasion success rates also tend to converge because the adversarial signal becomes too weak to dominate either training-time or inference-time behavior. Appendix Figure 2 provides the detailed plots.

9.2 Does the Choice of LLM Affect Attack Performance?

We also test whether the foundation LLM, rather than the graph encoding interface, explains attack performance. LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPH-PROMPTER uses Llama2-7B (Touvron et al. 2023). We replace LLAGA’s backbone with Llama2-7B and compare poisoning and evasion results across datasets. Clean accuracy remains similar (e.g., Vicuna: 0.89 $\pm$ 0.07, Llama2: 0.90 $\pm$ 0.03 on Cora), and attack vulnerability is comparable. The same trend holds when swapping LLMs in GRAPH-PROMPTER, suggesting that adversarial robustness is driven more by graph encoding design than by the specific foundation LLM. Appendix Table 2 reports the full results.

10 Defense: GALGUARD

The attack results motivate our defense contribution: graph-aware LLMs require an integrated defense because they are vulnerable through both structure and textual features. Existing GNN defenses are insufficient in this setting. Many defenses prune or reweight edges using node feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), which assumes that the features used to compute similarity are clean. When the features themselves are perturbed, these defenses can fail (see Figure 3 in the Appendix). Similarly, defenses that decouple structure and features (Shanthamallu, Thiagarajan,

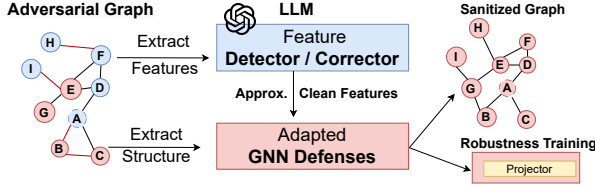


Figure 4: **Overview of GALGUARD.** The defense contribution addresses the two vulnerability channels identified in this paper: perturbed node features are shown in blue and perturbed edges in red. The LLM corrector first detects and repairs feature perturbations extracted from an adversarial graph. The corrected features and perturbed edges are then passed to adapted GNN defenses, which either sanitize the graph or support robustness training for graph-aware LLM projectors.

and Spanias 2021; Wu et al. 2021), as well as OCR-based text sanitization (Boucher et al. 2022), are not designed for coordinated feature-and-structure attacks. This gap is the rationale for GALGUARD.

We propose graph-aware LLM defense (GALGUARD), an end-to-end defense strategy with two complementary components: (1) an LLM-based feature corrector that targets imperceptible textual perturbations and (2) adapted GNN-based structural defenses that target adversarial edge perturbations. The design mirrors the contribution structure of the attacks: feature robustness and structural robustness must be handled together.

Feature Robustness Strategy. As shown in Figure 4, the feature component uses an LLM as an adaptive textual validator. It analyzes node features for anomalous patterns and generates approximate corrections that restore feature integrity, thereby mitigating feature-level perturbations before the graph-aware LLM consumes them. This module operationalizes the paper’s feature-attack finding as a practical defense step. Appendix A.6 provides the prompts and instructions used for the corrector. In our experiments, GPT-4 Turbo serves as the LLM corrector.

Graph Structure Robustness Strategy. The structural component adapts GNN defenses to the projector-based graph-aware LLM setting. First, we use a *graph purification heuristic* (Wu et al. 2019) as preprocessing: for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and neighboring embeddings $\phi(x_u)$, then remove low-similarity edges. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). For GRAPHPROMPTER, GNNGuard can be applied directly to the message-passing layers. For LLAGA, we adapt the defense to the node-template interface by incorporating purification into computational-tree construction and adding a learnable *global structural context embedding* (M_{global}) to the projector. Concatenating M_{global} with the purified node sequence embedding lets the model train on potentially perturbed graph inputs. Because GALGUARD is plug-and-play, other structural defenses, such as Pro-GNN (Jin et al. 2020), can replace GNNGuard.

10.1 Results

Table 6 evaluates the defense contribution under METAATTACK using accuracy. We compare the attacked models with GALGUARD_p, which combines the LLM feature corrector with graph purification, and the full defense, which adds GNNGuard or its adaptation. Across Cora, CiteSeer, and PubMed, and across poisoning and evasion settings, both defense variants consistently reduce the impact of METAATTACK. GALGUARD_p shows that feature correction plus initial graph purification is already a useful foundation. The full GALGUARD yields stronger robustness than GALGUARD_p, demonstrating that feature correction and GNN-inspired structural defense are complementary rather than redundant.

Table 6: Defense performance under METAATTACK poisoning and evasion. GALGUARD_p combines LLM-based feature correction with graph purification, while GALGUARD adds GNNGuard or its projector-specific adaptation.

Data	Method	LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper contributes the first systematic adversarial robustness study of learned-projector graph-aware LLMs. The central finding is that robustness depends on the graph encoding interface, not only on the underlying LLM. We show that poisoning and evasion attacks designed for GNNs remain relevant, with evasion especially damaging. We also show that encoding choices change vulnerability: the node sequence template in LLAGA significantly increases susceptibility, while GNN-based encoding is comparatively more robust. We further identify a new template injection attack surface in LLAGA and show that imperceptible textual feature perturbations can outperform structural attacks.

To complement these diagnostic contributions, we propose GALGUARD, an end-to-end defense that combines LLM-based feature correction with adapted GNN-based structural defenses. The broader implication is that graph-aware LLM security cannot be evaluated by reusing GNN robustness assumptions alone: these models require defenses that jointly consider textual features, graph structure, and the encoding mechanism that connects them. As graph-aware LLMs move toward applications in social networks, healthcare, finance, and other sensitive domains, this contribution provides a foundation for future robustness research and for practical safeguards before deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.

- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language.

In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.

Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.

Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.

Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.

Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.

Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.

Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.

Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.

Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.

Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.

Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.

Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.

Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.

Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.

Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.

Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.

Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix expands the contribution structure used in the main paper. Appendix A.1 provides the full graph-encoding taxonomy that motivates our focus on learned-projector graph-aware LLMs. Appendix A.2 reviews GNN attacks and explains the basis for transferring NETTACK and METAATTACK. Appendix A.3 gives implementation-level details for those attacks, while Appendix A.4 specifies the LLAGA node sequence injection algorithm. Appendix A.5 reports ablations on feature-perturbation level and foundation LLM choice. Appendix A.7 provides feature-attack examples and the LLM corrector prompt, and Appendix A.8 discusses implications for attacks and defenses.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The taxonomy is a supporting contribution because it makes the paper’s robustness scope precise. Existing graph-aware LLM methods differ primarily in how graph structure is encoded before reaching the language model. We group them into two categories: *textual descriptions of graphs*, where graph information is encoded as natural language, and *learned graph projector*, where a projector maps graph data into embeddings or structured forms that the LLM can process directly. Figure 5 illustrates these categories, and Table 7 summarizes their design differences.

Graph Representation as a Textual Description The first category represents graph data as natural language, so the LLM processes graph structure through textual descriptions rather than a learned graph encoder. This approach has been explored in work that converts graph data into textual formats and pairs the text with task-specific instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). For example, Tan et al. (2023) generate random-walk textual sequences for attributed graph embedding and use them to fine-tune LLMs. Zhao et al. (2023) translate graph structures into natural language to support graph reasoning in text space. Wei et al. (2024a) add visual graph representations to textual graph descriptions, improving LLM reasoning over graph data. In multimodal applications, Liu et al. (2023) and Su et al. (2022) link molecular graphs with text to support molecular retrieval and drug-discovery tasks. Huang et al. (2023a) study graph structure in LLM prompts and find that LLMs often process graph prompts as contextual paragraphs rather than explicit graph objects, underscoring the role of prompt design in systems such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Complementary work uses LLM-generated explanations as graph features to improve performance and interpretability (He et al. 2024).

A related textual-description line integrates LLM pretraining strategies with GNNs to improve graph reasoning. For instance, Chen et al. (2023a) propose using LLMs to enhance node features for node classification and using LLMs directly as predictors for graph tasks. Chandra et al. (2020) combine

LLM text encoders with GNN structure encoders for online fake-news detection. Chen et al. (2023b) use LLMs to annotate nodes for label-free node classification, after which the annotations train a GNN. Similarly, Mavromatis et al. (2023) use an LLM as a feature extractor for GNNs. These methods show the breadth of LLM support for graph learning, but their robustness interface remains primarily textual or GNN-mediated.

For inference, Zhu et al. (2024) propose a textual-graph fine-tuning method that combines LLMs and GNNs through tunable side structures alongside each LLM layer, improving training efficiency and inference speed. Duan et al. (2023) introduce SimTeG, where an LLM is first fine-tuned on a downstream task such as node classification, and node embeddings are then obtained from the final hidden states. These approaches are relevant background for graph-aware LLMs, but they also illustrate why textual-description methods can struggle to represent large or intricate graph structure within prompt limits.

Graph Representation as a Learned Projector The second category uses a learned projector to encode graph information before it reaches the LLM. This category is central to our robustness study because the projector creates a concrete graph-to-language interface where structural perturbations, textual feature perturbations, and model-specific template choices can interact. LLAGA (Chen et al. 2024), for example, uses node-level templates and a linear projector. GRAPH PROMPTER (Liu et al. 2024) uses a GNN-based projector to encode the graph. Other methods integrate GNN layers into pre-trained LLMs (Qin et al. 2023), use graph neural prompts to encode knowledge graphs (Tian et al. 2024), develop prompt-based node feature extractors with GNN adapters (Huang et al. 2023b), or compress graph information into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). These methods make graph-aware LLMs computationally efficient by freezing the LLM and training only the projector, but they also introduce graph-specific robustness questions.

This work. We focus on learned-projector graph-aware LLMs rather than textual-description methods (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Textual descriptions often cannot fully capture structural intricacies, especially for large graphs, because detailed graph descriptions quickly exceed prompt token limits and are better suited to simpler properties such as node degree. From a robustness perspective, they also do not expose the same graph-specific projector attack surfaces; when a graph is represented as text such as “Node A is connected to Node B,” conventional text adversarial attacks are the most direct threat. Learned-projector methods, by contrast, require graph encoders, node templates, and projectors, which motivates our analysis of structural attacks, feature perturbations, and LLAGA’s template-injection vulnerability in representative systems (Chen et al. 2024; Liu et al. 2024).

A.2 Attacks on GNN

This related-work appendix supports our transfer-study contribution by identifying which GNN attacks are relevant to

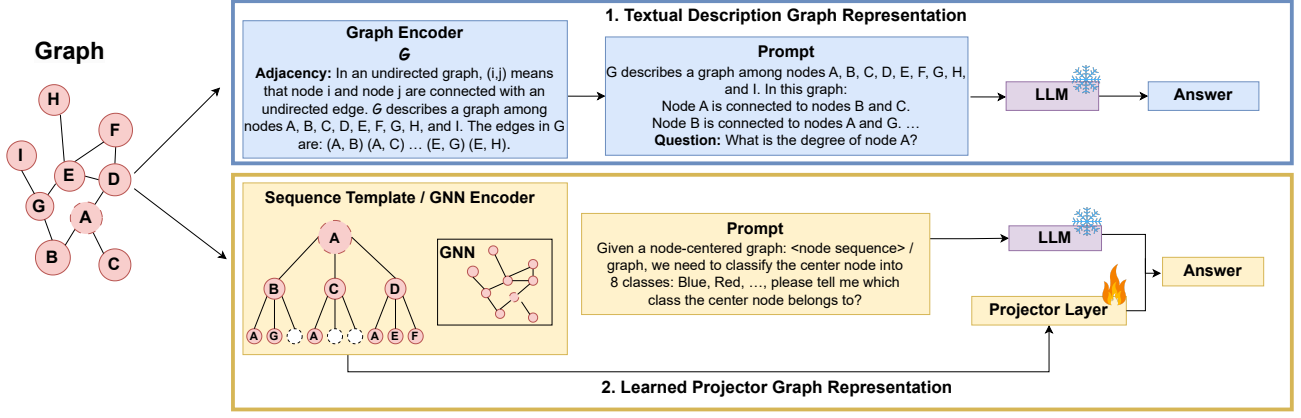


Figure 5: **Graph encoding-based adaptations of LLMs.** This expanded appendix version of the taxonomy distinguishes textual-description graph representations from learned-projector representations. Textual-description methods express graphs in natural language, whereas learned-projector methods encode graphs through node templates or GNNs before mapping them into LLM token space. In both learned-projector settings, the LLM is frozen and only the graph-to-token projector is trained.

graph-aware LLM robustness. Attacks on GNNs include membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which infer whether a node was in the training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which infer sensitive or missing node attributes from outputs or embeddings; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which infer global graph properties; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade performance through subtle graph or feature perturbations; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which reconstruct graph structure from model outputs or embeddings.

Among these categories, adversarial attacks are the focus of this paper because they directly compromise model integrity. In adversarial graph attacks, small crafted *perturbations* modify node features or graph edges to degrade predictive performance. These attacks are typically divided into **poisoning attacks**, which manipulate the training data, and **evasion attacks**, which perturb inputs at test time. Zügner, Akbarnejad, and Günnemann (2018) show that perturbing graph structure or node features can significantly degrade GNN performance; their method, **NETTACK**, greedily searches for minimal perturbations that maximize training loss and disrupt predictions. Li et al. (2023) revisit graph adversarial attacks from a data-distribution perspective and propose defenses for distributional shifts induced by adversarial examples. Wu et al. (2019) analyze graph adversarial examples and defenses, emphasizing how small perturbations can expose GNN vulnerabilities. **METAATTACK** (Zügner and Günnemann 2019) performs poisoning and evasion attacks by treating graph structure as an optimizable hyperparameter in a bi-level problem and using meta-gradients to select damaging edge modifications.

In the main paper, we use **NETTACK** (Zügner, Akbarnejad,

and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative attacks because together they cover targeted and untargeted graph perturbations as well as poisoning and evasion settings. This choice lets us ask a focused transfer question: whether attacks designed for GNNs remain effective when graph structure is routed through projector-based graph-aware LLMs.

A.3 How NETTACK and METAATTACK Work

This section supports the attack-transfer contribution by summarizing how the two representative GNN attacks operate before they are adapted to graph-aware LLMs. **NETTACK** perturbs graph structure by adding and removing edges while preserving the degree distribution of the perturbed graph. It uses greedy optimization with a surrogate model, specifically a linearized 2-layer graph convolutional network (GCN), to estimate how structural changes affect classification. Its objective is to increase confidence in incorrect predictions by maximizing the distance between logits on the perturbed and clean graphs. For each target node, **NETTACK** scores candidate edge perturbations and applies the highest-scoring add-or-remove operation. Because **NETTACK** is originally targeted, causing misclassification of a single node, we follow (Zügner and Günnemann 2019) and adapt it to an untargeted evaluation by randomly selecting test nodes, attacking them with **NETTACK** while considering all graph nodes, and aggregating the results.

METAATTACK directly optimizes graph structure by treating the adjacency matrix as a hyperparameter. It formulates the attack as a bi-level optimization problem: the inner problem minimizes training loss on the original graph, and the outer problem minimizes loss on the poisoned graph. Like **NETTACK**, **METAATTACK** uses a linearized two-layer GCN surrogate to approximate how structural perturbations affect model performance. A score function evaluates candidate perturbations, and the attack uses meta-gradients with respect to hyperparameters to determine which connected edges should

Table 7: Comparison of graph encoding-based adaptations of LLMs. The table makes the paper’s taxonomy contribution explicit by contrasting how textual-description and learned-projector methods encode graphs, modify the LLM, represent structure, and expose robustness-relevant tradeoffs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

be removed or which absent edges should be added. It then greedily selects the highest-scoring perturbations to maximize attack impact.

For both attacks, we keep the unnoticeability constraint from (Zügner, Akbarnejad, and Günnemann 2018), which limits changes to the graph degree distribution. Accordingly, all experiments perturb only 10% of the edges.

The adaptation to graph-aware LLMs follows the model-specific encoding interface. For poisoning attacks on LLAGA, we use the perturbed graph from NETTACK or METAATTACK to generate node-level sequences and retrain the projector that maps those sequences into token embeddings. For GRAPHPROMPTER, we use the perturbed graph to re-encode structure with a graph attention network (GAT) and retrain the projector. For evasion attacks, the graph-aware LLM is trained on clean data and receives perturbations only at inference time.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 formalizes the LLAGA template-injection contribution. LLAGA constructs a *fixed-shape* computational tree for each node u by sampling k neighbors per node up to depth d (lines 1–14). When a node has fewer than k sampled neighbors, the template inserts placeholders, and those placeholders become the attack surface. Our attack manipulates the graph so malicious nodes occupy these positions. *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* inserts one high-degree node into a placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* places distinct high-degree nodes across available placeholders (lines 35–43). The inserted nodes are uninformative for the target, so they corrupt the node sequence and degrade the learned projection

layer.

A.5 Ablation Studies

The ablations provide supporting evidence for two claims in the main paper: feature attacks scale with perturbation budget, and robustness is driven more by the graph encoding interface than by the specific foundation LLM.

Ablation Results: Varying Feature Perturbation Figure 6 reports how LLAGA and GRAPHPROMPTER respond as the percentage of perturbed textual features changes. Increasing the perturbation percentage consistently lowers accuracy for both models, confirming that imperceptible feature perturbations are not isolated failures but a budget-sensitive vulnerability channel.

Ablation Results: Does the Choice of LLM Affect Attack Performance? Table 8 supports the paper’s graph-encoding interpretation by showing that the base LLM has minimal effect on clean performance or adversarial vulnerability in the LLAGA framework. Across Cora, Citeseer, and PubMed, Vicuna-7B and Llama2-7B variants achieve similar clean accuracies and similar degradation under poisoning and evasion attacks. For example, under METAATTACK evasion on Cora, accuracy is 0.44 ± 0.06 with Vicuna-7B and 0.43 ± 0.08 with Llama2-7B, which lies within the reported standard deviations. Similar patterns across datasets and attack types indicate that attack effectiveness is largely independent of the foundation LLM.

A.6 Results on the ArXiv Dataset

The ArXiv experiment tests whether the node sequence template attack remains relevant on a larger graph. ArXiv contains 169,343 nodes and 1,166,243 edges, with sparsity 0.81.

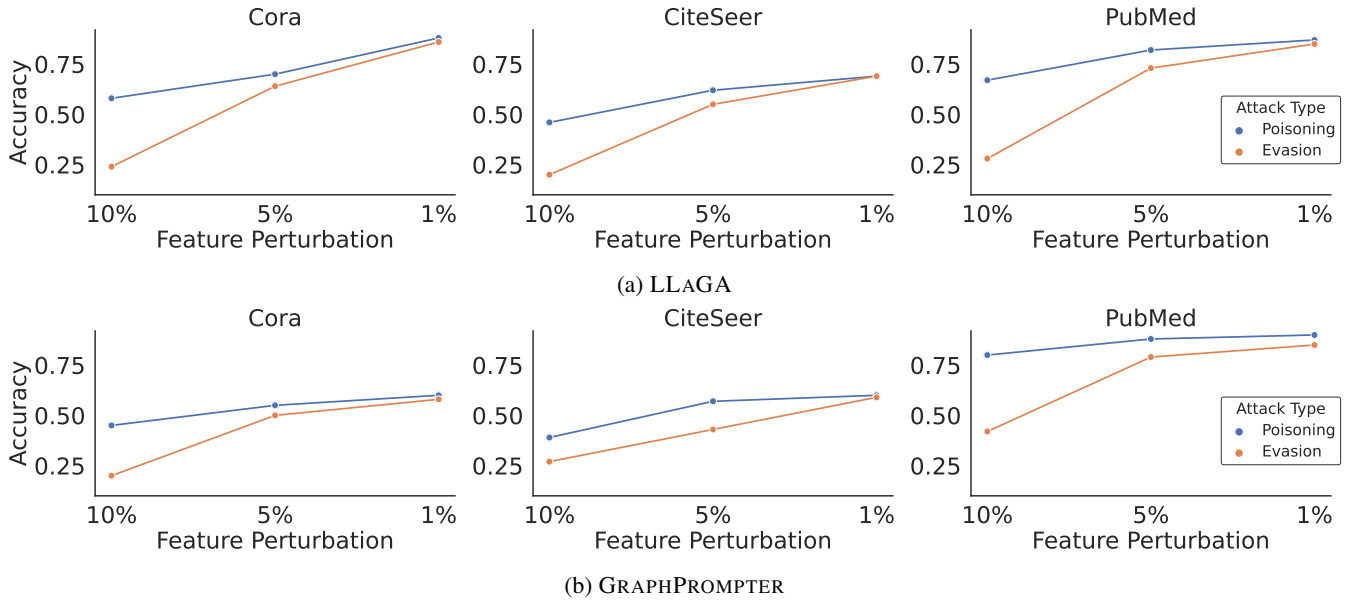


Figure 6: Sensitivity of graph-aware LLMs to feature-perturbation budget. Lower accuracy indicates a stronger attack as the percentage of perturbed textual features increases.

Table 8: Effect of changing the base LLM in LLAGA. Similar clean and attacked accuracies indicate that robustness is governed more by the graph encoding interface than by the selected foundation LLM.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

As shown in Table 9, poisoning and evasion behave differently on ArXiv than on Cora, Citeseer, and PubMed: poisoning attacks are consistently stronger than evasion attacks across all three injection strategies. This suggests that ArXiv’s structure may make the projector more vulnerable to training-time template corruption while remaining comparatively robust at inference time. Even with low sparsity, inserting a single supernode can substantially affect performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples in Appendix A.7 make the feature-attack contribution concrete using real Cora node features. The “input” is the text received by the LLM after the attack is applied. Although a human reader sees the original and perturbed text as visually identical or very similar, the LLM receives different Unicode characters or a different character order, which can change its behavior. Table 10 illustrates the homoglyph attack, and Table 11 illustrates the reordering attack.

Table 9: Node sequence template injection results including the large-scale ArXiv setting. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean reports accuracy without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	—	0.72 ± 0.03	—
	Poisoning	0.60 ± 0.04	0.58 ± 0.02	0.55 ± 0.03
	Evasion	0.65 ± 0.03	0.62 ± 0.02	0.60 ± 0.03

Table 10: Imperceptible homoglyph attack example. The perturbed text is processed as different characters by the LLM, while the rendered text appears nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Imperceptible reordering attack example. The perturbed text is processed in a different character order by the LLM, while the rendered text remains nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Feature Corrector Defense Template The feature corrector implements the defense contribution against imperceptible textual perturbations. It is a prompt-based LLM module designed to detect and correct homoglyph substitutions and character reorderings before graph-aware LLM inference. The prompt asks the LLM to act as a linguistic validator, focusing on subtle perturbations that are difficult for a human reader to notice but can change the token sequence processed by the model.

The prompt template below specifies the task and output format so the LLM returns a corrected version of the input text when perturbations are detected and leaves clean text unchanged.

A.8 Detailed Discussion

This discussion section restates the main contributions in reviewer-facing terms and highlights their implications. The experiments show that graph-aware LLMs should not be evaluated only as stronger graph predictors; they should also be evaluated as systems with new adversarial interfaces. Across LLGA and GRAPHPROMPTER, attacks transfer from GNNs, new vulnerabilities arise from graph encoding, feature perturbations become unusually salient, and defenses must address both structure and text.

Impact of Graph Encoding on Adversarial Vulnerability The first design implication is that graph encoding choices shape adversarial vulnerability. LLGA’s node sequence template improves graph-structure integration, but it also introduces a template-specific attack surface that can be exploited by injecting malicious placeholder nodes. This vul-

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: The prompt template used for the LLM-based feature corrector module. It explicitly defines the role, task, and expected output format for the LLM.

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k
Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow 2\text{-hop neighborhood of } u$ 
21:     if AttackStrategy = NI then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if AttackStrategy = SI then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:       break ▷ Only use first placeholder
34:     end for
35:     else if AttackStrategy = MSI then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

nerability is not a generic GNN failure mode; it is created by the graph-to-LLM interface. GRAPHPROMPTER, which uses a GNN-based projector, is comparatively more resilient in

our experiments, suggesting that hybrid graph encoders can buffer some structural perturbations. The broader contribution is therefore not only that one model is more vulnerable than another, but that robustness evaluation must inspect how each graph-aware LLM encodes graph structure.

Sensitivity of Graph-aware LLMs to Feature Perturbations The second implication is that textual node features are a major adversarial channel. Despite using different structural encoders, both LLGA and GRAPHPROMPTER are sensitive to imperceptible feature perturbations. This result extends the GNN robustness conversation: attacks that are often secondary in traditional graph settings become central when graph-aware LLMs rely on text. Subtle Unicode substitutions or reorderings can degrade performance while remaining hard for humans to notice. Text sanitization methods such as those discussed by (Boucher et al. 2022) offer one defense direction, but homoglyph and reordering attacks remain difficult to eliminate. Our LLM-based corrector is a practical mitigation that asks the model to detect and repair suspicious inputs before downstream graph-aware inference.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs The third implication is that attack effectiveness depends on dataset structure. Cora, Citeseer, PubMed, and ArXiv do not respond identically to poisoning, evasion, or template injection. This variability matters because graph-aware LLMs are likely to be deployed across domains with different graph sizes, sparsity levels, and feature distributions. Robustness claims should therefore be evaluated across datasets rather than inferred from a single benchmark.

Implications for Graph-aware LLM Development The development implication is that graph-aware LLM design must balance predictive performance with attack-surface control. As these models take on tasks traditionally handled by GNNs, graph encoders, node templates, projectors, and textual feature pipelines should be treated as security-relevant components. Understanding when GNN attacks transfer, when LLM-style feature perturbations dominate, and when model-specific graph encoders create new vulnerabilities opens a research path for robust graph-aware LLM architectures and defenses.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs Defenses on adversarial attacks based on feature perturbation. Existing GNN defenses are not designed for the feature-attack contribution identified in this paper. GNN adversarial work has largely emphasized structural perturbations because feature perturbations often fail to change the target node’s predicted class (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019). As a result, there are *no known defenses against feature-based attacks on GNNs* that directly address our setting. Graph-aware LLMs change this assumption: feature perturbations that were previously weak can now manipulate model behavior through the textual input channel. OCR-style sanitization may help, but it is imperfect and can misread homoglyphs more often than unperturbed text (Boucher et al. 2022). This gap motivates our LLM-based

self-correction module for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based attacks transfer from GNNs to graph-aware LLMs, especially under evasion, it is tempting to reuse GNN structural defenses directly. Our results show why this is insufficient. Unlike the traditional GNN setting, *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective* when the defense relies on clean features.

A common GNN defense strategy makes the adjacency matrix trainable or adjusts edge weights so that edges between dissimilar nodes receive lower weights and edges between similar nodes receive higher weights. Preprocessing defenses build on this idea by removing edges between low-similarity nodes (Wu et al. 2019) or randomly dropping edges (Dai et al. 2018). These methods depend on clean node features to compute reliable similarity scores. When node features are perturbed, the same mechanisms can remove useful edges or introduce incorrect ones. We illustrate this failure on a random Cora node in Figure 8: clean feature similarity preserves some original edges, whereas perturbed feature similarity preserves none of the original connections and reconstructs incorrect edges.

A second defense family reduces reliance on graph structure. For example, (Shanthamallu, Thiagarajan, and Spanias 2021) train a surrogate predictor using only node features and use uncertainty matching to extract graph information from the GNN. Wu et al. (2021) train separate feature and structural submodels and distill knowledge by adding confident unlabeled data to the training set. These defenses also assume clean node features, which does not hold in our coordinated feature-and-structure attack setting. Therefore, structural defenses developed for GNNs are not directly sufficient for graph-aware LLMs, motivating integrated defenses that combine feature correction with structural protection.

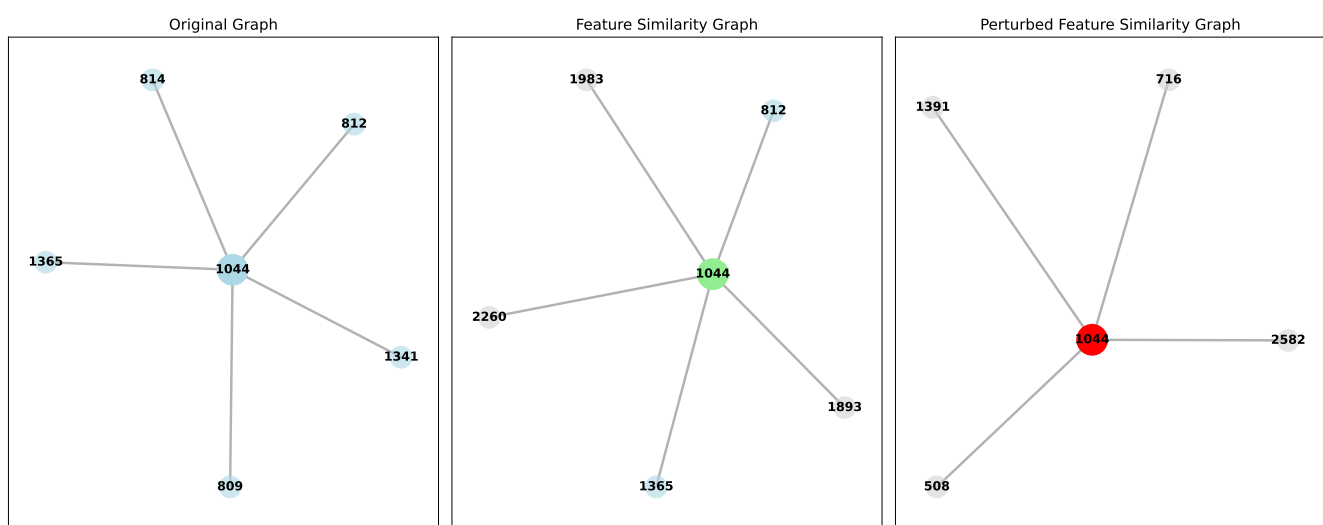


Figure 8: Feature-similarity defense failure under perturbed node text for random node 1044. **Original (left)** shows the ground-truth graph edges. **Feature Similarity (middle)** reconstructs edges using clean node features, with blue nodes indicating preserved original connections. **Perturbed Feature Similarity (right)** reconstructs edges using perturbed node features and fails to preserve the original connections.